Lecture Notes: When Models Surprise Us

Scaling Laws, Emergent Abilities, Chain-of-Thought,
and the Test-Time Compute Revolution

LLM Lab — Executive Education Program

2026

Contents

1 Introduction: The Unifying Thread **3**

2 Topic A: Phase Transitions in Capabilities **3**
2.1 Background: Neural Scaling Laws . . . 3
2.1.1 Kaplan et al. (2020): Power-Law Scaling . . . 3
2.1.2 Hoffmann et al. (2022): The Chinchilla Scaling Law . . . 3

3 Topic B: Emergent Abilities of Large Language Models **4**
3.1 The Core Claim: Wei et al. (TMLR 2022) . . . 4
3.1.1 Scale of Emergence . . . 4
3.2 Concrete Examples of Emergent Abilities . . . 4
3.2.1 Arithmetic and Symbolic Reasoning . . . 5
3.2.2 Multi-Step Reasoning . . . 5
3.2.3 Language Understanding and Knowledge . . . 6
3.2.4 Code and Instruction Following . . . 6
3.2.5 The "Capability Ladder": A Synthesis . . . 6
3.3 The Physics Analogy: Anderson's "More Is Different" (1972) . . . 7
3.4 The Mirage Rebuttal: Schaeffer et al. (NeurIPS 2023) . . . 7
3.4.1 The Core Mathematical Argument . . . 8
3.4.2 Resolution Effects . . . 8
3.4.3 The BIG-Bench Meta-Analysis . . . 8
3.4.4 Three Predictions Tested on GPT-3/InstructGPT . . . 8
3.4.5 Vision Experiments . . . 8
3.5 The Nuanced Resolution . . . 9

4 Topic C: Chain-of-Thought and the Rise of Thinking Models **9**
4.1 The Discovery: Chain-of-Thought Prompting . . . 9
4.1.1 Wei et al. (NeurIPS 2022): Few-Shot Chain-of-Thought . . . 9
4.1.2 Kojima et al. (NeurIPS 2022): Zero-Shot Chain-of-Thought . . . 10
4.2 Scaling Chain-of-Thought: Self-Consistency and Tree of Thoughts . . . 10
4.2.1 Self-Consistency (Wang et al., ICLR 2023) . . . 10
4.2.2 Tree of Thoughts (Yao et al., NeurIPS 2023) . . . 11
4.2.3 Least-to-Most Prompting (Zhou et al., ICLR 2023) . . . 11
4.3 From Prompting to Training: The Emergence of Thinking Models . . . 11
4.3.1 Two Training Paradigms . . . 11
4.3.2 OpenAI o1 (September 2024) . . . 12

4.3.3 DeepSeek-R1 (January 2025): Reasoning from Pure RL . . . 12
4.4 Budget Forcing: Controlling How Long a Model Thinks . . . 13
4.4.1 The s1K Dataset . . . 13
4.4.2 How Budget Forcing Works . . . 14
4.4.3 Results . . . 14
4.5 Taxonomy of Test-Time Compute Methods . . . 14
4.5.1 The Snell et al. Framework (ICLR 2025) . . . 14
4.5.2 Full Taxonomy . . . 15
4.5.3 Process Reward Models . . . 15
4.5.4 The System 1 / System 2 Analogy . . . 15
4.5.5 Two Axes of Scaling: Strategic Implications . . . 15

5 Topic D: Grokking **16**
5.1 The Discovery: Power et al. (2022) . . . 16
5.1.1 Experimental Setup . . . 16
5.1.2 Training Dynamics (Division mod 97, 50% data) . . . 16
5.1.3 Which Operations Grokked . . . 16
5.1.4 The Role of Regularisation . . . 16
5.2 Mechanistic Explanation: Nanda et al. (ICLR 2023) . . . 16
5.2.1 Setup . . . 17
5.2.2 The Learned Algorithm: Discrete Fourier Transform . . . 17
5.2.3 Quality of the Approximation . . . 17
5.2.4 Three Training Phases . . . 17
5.2.5 Progress Measures . . . 18
5.2.6 The Role of Weight Decay . . . 18
5.3 Circuit Efficiency: Varma et al. (ICLR 2025) . . . 18
5.3.1 The Efficiency Framework . . . 18
5.3.2 Three Conditions for Grokking . . . 18
5.3.3 Ungrokking . . . 19
5.3.4 Semi-Grokking . . . 19
5.4 Grokking at Scale: Li et al. (2025) . . . 19
5.4.1 Model and Setup . . . 19
5.4.2 Local Grokking . . . 19
5.4.3 Pathway Evolution Metrics . . . 19
5.5 Connection to Classical Learning Theory . . . 19
5.5.1 Violation of the Bias-Variance Tradeoff . . . 19
5.5.2 Connection to Double Descent . . . 20

6 Synthesis: What the Audience Should Take Away **20**

1 Introduction: The Unifying Thread

These notes accompany the presentation series "When Models Surprise Us." They provide the technical depth behind each topic, suitable for a presenter with a master's-level background in computer science or machine learning.

The topics share a single message: **the dynamics of learning in large language models violate our deepest intuitions about how systems acquire capabilities.**

1. **Scaling laws:** Loss improves smoothly and predictably with scale — but downstream task accuracy does not.
2. **Emergent abilities:** Scaling a model can produce discontinuous jumps in what it can do — but some of these jumps may be measurement artifacts.
3. **Chain-of-thought and thinking models:** Making models "show their work" dramatically improves reasoning, and training models to reason has produced an entirely new paradigm of test-time compute scaling.
4. **Grokking:** A model can memorise its training data perfectly, appear to stop learning, and then — much later — suddenly generalise.

2 Topic A: Phase Transitions in Capabilities

2.1 Background: Neural Scaling Laws

Before discussing emergence, we must understand the baseline expectation: *smooth, predictable scaling.*

2.1.1 Kaplan et al. (2020): Power-Law Scaling

Kaplan et al. [2] established that cross-entropy loss follows power-law relationships with model size N (non-embedding parameters), dataset size D (tokens), and compute C (FLOPs):

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad N_c = 8.8 \times 10^{13},\ \alpha_N = 0.076 \tag{1}$$

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad D_c = 5.4 \times 10^{13},\ \alpha_D = 0.095 \tag{2}$$

$$L(C) = \left(\frac{C_{\min}}{C}\right)^{\alpha_C}, \quad C_{\min} = 3.1 \times 10^{8},\ \alpha_C = 0.050 \tag{3}$$

These hold when each variable is scaled independently without the others becoming a bottleneck. The key implication: multiplying parameters by 10 reduces loss by a factor of $10^{-0.076} \approx 0.84$ (a 16% reduction). Loss improves smoothly and predictably.

Kaplan concluded that compute-optimal allocation should scale parameters much faster than data: $N_{\text{opt}} \propto C^{0.73}$, $D_{\text{opt}} \propto C^{0.27}$. This was later corrected by Hoffmann et al.

2.1.2 Hoffmann et al. (2022): The Chinchilla Scaling Law

Hoffmann et al. [3] proposed a parametric loss function:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} \tag{4}$$

where E is the irreducible entropy of natural language. Their fitted constants:

$$E = 1.69, \quad A = 406.4, \quad \alpha = 0.34, \quad B = 410.7, \quad \beta = 0.28.$$

A replication by Besiroglu et al. [4] found somewhat different values ($E = 1.82$, $\alpha = 0.35$, $\beta = 0.37$) due to numerical issues in the original optimisation, but the qualitative conclusion held.

The critical correction of Kaplan: compute-optimal allocation should scale parameters and data *equally*:

$$N_{\text{opt}} \propto C^a, \quad D_{\text{opt}} \propto C^b, \quad a \approx b \approx 0.5. \tag{5}$$

This yields the famous "20 tokens per parameter" heuristic. Chinchilla (70B parameters, 1.4T tokens) matched the performance of Gopher (280B parameters, 300B tokens) with the same compute budget.

Remark 2.1. These scaling laws describe **loss** (cross-entropy on next-token prediction), which improves smoothly. The next section concerns **downstream task accuracy**, which can behave very differently. This disconnect is the crux of the emergence debate.

3 Topic B: Emergent Abilities of Large Language Models

This section expands on one of the most debated phenomena in modern AI: the observation that certain capabilities appear to arise *discontinuously* as language models are scaled up.

3.1 The Core Claim: Wei et al. (TMLR 2022)

Definition 3.1 (Emergent Ability [5])**.** An ability is *emergent* if it is not present in smaller models but is present in larger models. Specifically, performance remains near-random for all models below some critical scale threshold, then jumps sharply to well-above-random. This emergence "cannot be predicted by extrapolating a scaling law based on small-scale models."

Wei et al. examined eight model families: GPT-3 (350M–175B), LaMDA (40M–137B), PaLM (8B–540B), Chinchilla (7B–70B), and Gopher (7.1B–280B), among others. The primary scaling variable is **training compute** (FLOPs), though parameter count is roughly proportional.

3.1.1 Scale of Emergence

The paper catalogued emergent abilities across two major evaluation suites:

- **BIG-Bench:** 67 out of ∼210 tasks showed emergent behaviour.
- **MMLU:** 51 tasks showed emergent behaviour.
- Wei later compiled 137 emergent abilities across the literature.

3.2 Concrete Examples of Emergent Abilities

What makes emergence striking is how specific and diverse the capabilities are. Below we catalogue concrete examples, organised by the type of ability that emerges.

3.2.1 Arithmetic and Symbolic Reasoning

Task	Model	Threshold	What happens
3-digit addition	GPT-3	∼13B	Below 13B: ∼0% exact match. Above: jumps to >80%. The model cannot add 3-digit numbers at all, then suddenly can.
Modular arithmetic	GPT-3	∼13B	Computing $(a + b) \bmod n$ — near-zero below threshold, then sharp jump.
Multi-digit multiplication	PaLM	∼62B	2-digit × 2-digit multiplication. Below 62B: random. Above: >50%.

Example 3.1 (3-Digit Addition)**.** Consider the task: "What is 456 + 789?" The correct answer is "1245." A 6.7B-parameter GPT-3 model outputs random-looking digits. A 175B-parameter GPT-3 model reliably outputs "1245." There is no intermediate regime where the model gets "close" — under exact-match scoring, it goes from 0% to >80% accuracy in a narrow range of model sizes.

3.2.2 Multi-Step Reasoning

Task	Model	Threshold	What happens
Chain-of-thought reasoning	LaMDA	∼68B	Below: CoT prompting *hurts* performance. Above: dramatic improvements on GSM8K, StrategyQA.
Logical deduction (5 objects)	PaLM	∼62B	"Alice is taller than Bob. Bob is taller than Carol... Who is shortest?" — random below, >60% above.
Navigate (spatial reasoning)	PaLM	∼62B	Following sequences of spatial instructions.

Example 3.2 (Chain-of-Thought as Emergent Ability)**.** Wei et al. [9] showed that chain-of-thought prompting (providing step-by-step reasoning exemplars) **only works above ∼100B parameters**. For a 422M LaMDA model, adding CoT exemplars to the prompt actually *decreases* accuracy — the small model produces fluent but logically incoherent reasoning chains that lead to wrong answers. For PaLM 540B, the same technique improves GSM8K accuracy from ∼18% to ∼57%. The ability to benefit from "thinking step by step" is itself an emergent ability.

3.2.3 Language Understanding and Knowledge

Task	Model	Threshold	What happens
MMLU (college exams)	GPT-3	∼175B	Performance on 57 college-level subjects. Below 175B: near random (25% on 4-choice). Above: >45%.
Word in Context (WiC)	PaLM	∼540B	Does a word have the same meaning in two sentences? Only the largest PaLM model exceeds chance.
TruthfulQA	Gopher	∼280B	Smaller models are *less* truthful than random due to learned misconceptions. Only at ∼280B does truthfulness improve.

Example 3.3 (TruthfulQA: Inverse Scaling Then Emergence)**.** TruthfulQA [6] is remarkable because it exhibits **inverse scaling** followed by emergence. Medium-sized models (6B–70B) are *less truthful* than small models because they have learned common misconceptions from training data (e.g., "We only use 10% of our brains"). Only at very large scale (∼280B+) does truthfulness improve, as the model acquires sufficient world knowledge to distinguish fact from popular fiction.

3.2.4 Code and Instruction Following

Task	Model	Threshold	What happens
HumanEval (coding)	GPT-3/Codex	∼12B	Generate correct Python functions from docstrings. Below 12B: <5% pass@1. Codex-12B: ∼28%.
Instruction following	GPT-3	∼175B	Following novel instructions zero-shot (e.g., "Translate to French and then summarise"). Smaller models ignore or misinterpret compound instructions.

3.2.5 The "Capability Ladder": A Synthesis

Across all documented examples, a rough hierarchy of emergence thresholds appears:

Scale	Capabilities that emerge
∼1B	Basic pattern completion, simple factual recall
∼10B	Arithmetic (2–3 digits), basic code generation, simple analogies
∼60–100B	Multi-step reasoning, chain-of-thought, logical deduction, modular arithmetic
∼100–175B	College-level exam performance (MMLU), complex instruction following
∼500B+	Word sense disambiguation, nuanced truthfulness, advanced calibration

Remark 3.1. These thresholds are approximate and depend on the model family, training data, and evaluation protocol. Different architectures may exhibit different thresholds for the same capability. The key observation is the **qualitative pattern**: capabilities cluster at certain scale ranges rather than appearing uniformly.

3.3 The Physics Analogy: Anderson's "More Is Different" (1972)

Wei et al. explicitly invoke the framework of P.W. Anderson's seminal essay [1]. Anderson argued that **reductionism does not imply constructionism**: knowing the fundamental laws governing a system's components does not allow you to predict the emergent properties of the system at scale.

Anderson's central mechanism is **broken symmetry**: the solutions to equations can have lower symmetry than the equations themselves. His hierarchy:

X (higher level)	Y (lower level)
Solid-state physics	Elementary particle physics
Chemistry	Many-body physics
Molecular biology	Chemistry
Cell biology	Molecular biology
Psychology	Physiology
Social sciences	Psychology

Key examples from Anderson:

- **Ammonia (NH_3):** Tunnels between two mirror-image states $\sim 3 \times 10^{10}$ times per second. For larger molecules, tunnelling times exceed the age of the universe — symmetry is effectively broken by scale alone.
- **Crystallisation:** A gas with continuous translational symmetry spontaneously forms a crystal lattice with discrete symmetry. Rigidity is an emergent property.
- **Superconductivity:** Broken gauge symmetry (BCS theory). The fundamental equations were known for 30 years before the phenomenon could be explained.

The analogy to LLMs: knowing the transformer architecture and training loss does not, in Anderson's framework, allow you to predict what capabilities emerge at what scale — just as knowing quantum mechanics does not allow you to predict superconductivity from first principles.

3.4 The Mirage Rebuttal: Schaeffer et al. (NeurIPS 2023)

Schaeffer, Miranda, and Koyejo [7] (Outstanding Paper Award) argued that emergent abilities are **not a property of the model, but of the researcher's choice of evaluation metric.**

3.4.1 The Core Mathematical Argument

Suppose per-token probability of correctness $p(N)$ improves smoothly with scale N. For a task requiring a correct output sequence of length L, the **Exact String Match** metric scores:

$$\text{Accuracy}(N) = p(N)^L \tag{6}$$

This exponential creates artificial sharpness:

Per-token accuracy p	Sequence length L	Exact-match accuracy p^L
60%	4	13.0%
80%	4	40.9%
90%	4	65.6%
98%	4	92.2%

The improvement from $p = 0.6$ to $p = 0.98$ is smooth and gradual, but the exact-match metric transforms it into an apparent phase transition (13% $\rightarrow$ 92%).

3.4.2 Resolution Effects

With a test set of size T, the smallest measurable nonzero performance is $1/T$. Any true performance below $1/T$ reads as exactly zero, creating an artificial plateau that makes the eventual rise appear more sudden.

3.4.3 The BIG-Bench Meta-Analysis

More than 92% of claimed emergent abilities in BIG-Bench appeared under just two metrics: Multiple Choice Grade and Exact String Match. Both are discontinuous. Of the 39 metrics available in BIG-Bench, emergent abilities appeared with at most 5 of them.

3.4.4 Three Predictions Tested on GPT-3/InstructGPT

Schaeffer et al. tested three predictions using arithmetic tasks (2-shot multiplication of two 2-digit integers, addition of two 4-digit integers):

1. **Changing the metric:** Switching from Exact String Match to Token Edit Distance made the sharp transition disappear — performance improved smoothly and predictably. *Confirmed.*
2. **Increasing test set size:** Improving measurement resolution smoothed out apparent emergence even under nonlinear metrics. *Confirmed.*
3. **Varying sequence length:** Under Exact String Match, accuracy fell geometrically with L (as predicted by Eq. 6). Under Token Edit Distance, the decrease was approximately quasi-linear. *Confirmed.*

3.4.5 Vision Experiments

To demonstrate that emergence is not special to language models, they *induced* apparent emergent abilities in vision models (CNNs, autoencoders, Vision Transformers) on CIFAR-100:

- Under smooth metrics (Brier Score, mean per-class recall): smooth scaling.
- Under thresholded metrics analogous to Exact String Match: apparent sharp transitions.

No vision researcher has ever claimed emergent abilities — precisely because the vision community does not use harsh all-or-nothing metrics.

3.5 The Nuanced Resolution

The truth is that **both sides are partly right**:

1. **Some emergence is metric artifact:** Schaeffer's argument is mathematically rigorous and empirically confirmed for many BIG-Bench tasks.
2. **Some emergence may be real:** Tasks with composite structure (requiring k correct sub-steps) have success rates $\sim p^k$, which creates genuine threshold behaviour even under continuous per-step metrics. Multi-step reasoning tasks may exhibit real phase transitions.
3. **The unresolved question:** Whether *all* apparent emergence can be explained by metric choice and finite test-set resolution, or whether some phenomena are genuinely discontinuous, remains actively debated. The Berti et al. (2025) survey [8] provides the most balanced current assessment.

Key message: We genuinely cannot predict what the next model will be able to do. Both the optimists and the sceptics are partly right — which makes the uncertainty irreducible and strategically important.

4 Topic C: Chain-of-Thought and the Rise of Thinking Models

This section traces one of the most consequential developments in modern AI: the discovery that making language models "show their work" dramatically improves their reasoning, and how this insight evolved from a simple prompting trick into an entirely new paradigm for building AI systems.

4.1 The Discovery: Chain-of-Thought Prompting

4.1.1 Wei et al. (NeurIPS 2022): Few-Shot Chain-of-Thought

Wei et al. [9] demonstrated that including intermediate reasoning steps in few-shot exemplars dramatically improves performance on reasoning tasks — without any fine-tuning or architectural changes.

The prompting format. Standard few-shot prompting:

```
Q: Roger has 5 tennis balls.  He buys 2 more cans of tennis balls.  Each
can has 3 tennis balls.  How many does he have now?
A: 11
```

Chain-of-thought prompting:

```
Q: Roger has 5 tennis balls.  He buys 2 more cans of tennis balls.  Each
can has 3 tennis balls.  How many does he have now?
A: Roger started with 5 balls.  2 cans of 3 tennis balls each is 6 tennis
balls.  5 + 6 = 11.  The answer is 11.
```

The only difference is the inclusion of intermediate steps. Wei et al. used **8 hand-written exemplars** per benchmark.

Models tested: Five families at multiple scales — GPT-3 (350M–175B), LaMDA (422M–137B), PaLM (8B–540B), UL2 (20B), and Codex.

Tasks: Three categories:

- **Arithmetic reasoning:** GSM8K, SVAMP, ASDiv, AQuA, MAWPS
- **Commonsense reasoning:** CommonsenseQA, StrategyQA

- **Symbolic reasoning:** Last Letter Concatenation, Coin Flip

Key results:

Method	GSM8K	StrategyQA
PaLM 540B (standard few-shot)	∼18%	∼63%
PaLM 540B (chain-of-thought)	∼58%	75.6%
Prior SOTA (fine-tuned GPT-3 + verifier)	55%	69.4%

A prompting-only technique, requiring zero additional training, surpassed a heavily engineered fine-tuned system.

Remark 4.1. CoT gains were negligible or **negative** for small models. For models below ∼10B parameters, adding chain-of-thought exemplars actually *hurt* performance — the models produced fluent but logically incoherent reasoning. The ability to benefit from "thinking step by step" is itself an emergent ability (see Section 3).

4.1.2 Kojima et al. (NeurIPS 2022): Zero-Shot Chain-of-Thought

Kojima et al. [10] made a striking discovery: simply appending **"Let's think step by step."** to the prompt — with no exemplars at all — triggers chain-of-thought reasoning.

Two-stage process:

1. **Reasoning extraction:** Append "Let's think step by step." The model generates a free-form reasoning trace.
2. **Answer extraction:** Feed the question + generated reasoning back with "Therefore, the answer is" to extract a clean final answer.

Key results (InstructGPT text-davinci-002):

Benchmark	Zero-shot	Zero-shot CoT
MultiArith	17.7%	78.7%
GSM8K	10.4%	40.7%

A single, task-agnostic sentence produces most of the benefit of carefully crafted few-shot exemplars. This demonstrated that reasoning capabilities are **latent** in large language models and can be triggered by remarkably simple instructions.

4.2 Scaling Chain-of-Thought: Self-Consistency and Tree of Thoughts

4.2.1 Self-Consistency (Wang et al., ICLR 2023)

Wang et al. [11] observed that complex reasoning problems typically admit multiple valid solution paths. Their method:

1. Prompt with CoT exemplars (same as Wei et al.).
2. Instead of greedy decoding, **sample N diverse reasoning paths** at temperature > 0.
3. **Majority vote** over the final answers (marginalise out the reasoning paths).

The intuition: correct reasoning paths converge on the unique correct answer, while incorrect paths disagree with each other.

Improvements over standard CoT:

Benchmark	Improvement
GSM8K	+17.9%
SVAMP	+11.0%
AQuA	+12.2%
StrategyQA	+6.4%
ARC-challenge	+3.9%

Self-consistency requires **no additional training** — only multiple forward passes through the same model. It is a pure decoding-time strategy that trades compute for accuracy.

4.2.2 Tree of Thoughts (Yao et al., NeurIPS 2023)

Yao et al. [12] generalised CoT from a single linear chain to a **tree structure**:

1. **Thought decomposition:** Break reasoning into intermediate "thought" units (e.g., a single equation, one line of a plan).
2. **Thought generation:** At each node, generate multiple candidate next thoughts.
3. **State evaluation:** Use the LLM itself to evaluate partial reasoning states ("Is this approach promising?").
4. **Search:** Traverse the tree using BFS (breadth-first, expand all, prune to top-k) or DFS (depth-first, backtrack when unpromising).

Key result (GPT-4): Game of 24 (find arithmetic operations on 4 numbers to make 24): **4%** with CoT → **74%** with Tree of Thoughts. An 18.5× improvement from organising reasoning as search rather than a single chain.

4.2.3 Least-to-Most Prompting (Zhou et al., ICLR 2023)

Zhou et al. [13] addressed the "easy-to-hard generalisation" gap: standard CoT struggles when test problems are harder than the few-shot exemplars.

Two-stage approach:

1. **Decomposition:** Prompt the LLM to break a complex problem into simpler subproblems (from "least" to "most" difficult).
2. **Sequential solving:** Solve each subproblem in order, with answers to previous subproblems included in context.

Key result: On SCAN (compositional generalisation, length split), GPT-3 code-davinci-002 with least-to-most prompting achieved ≥**99%** accuracy using 14 exemplars. Standard CoT achieved only **16%** on the same split.

4.3 From Prompting to Training: The Emergence of Thinking Models

CoT prompting (2022) showed that making models "show their work" dramatically improved reasoning. This raised a natural question: **can we train models to natively generate reasoning traces, rather than relying on prompting?**

4.3.1 Two Training Paradigms

Approach 1: Supervised Fine-Tuning (SFT) on reasoning traces. Collect or generate step-by-step reasoning data, then fine-tune the model to produce similar traces. The model learns to *imitate* human-like reasoning.

Approach 2: Reinforcement Learning (RL). Train the model with a reward signal based on answer correctness. The model discovers its *own* reasoning strategies through trial and error, without seeing human reasoning demonstrations.

The second approach proved transformative.

4.3.2 OpenAI o1 (September 2024)

OpenAI o1 was the first major "reasoning model" with a **hidden internal chain of thought**:

- Trained via RL to generate extended reasoning tokens before answering.
- Reasoning tokens are generated but **hidden from the user** (API users are billed for them but cannot read them).
- Up to 32,768 output tokens (o1-preview), with ∼25,000 allocated for reasoning.

Benchmark results:

Benchmark	GPT-4o	o1-preview
AIME 2024 (competition maths)	∼12%	74%
Codeforces (competitive programming)	Elo 808 (11th %ile)	Elo 1807 (89th %ile)
GPQA Diamond (PhD-level science)	∼50%	78.1%

The model achieves expert-level performance on tasks where standard LLMs fail, purely by being trained to "think" before responding.

4.3.3 DeepSeek-R1 (January 2025): Reasoning from Pure RL

DeepSeek-R1 [16] demonstrated that reasoning can emerge from **pure reinforcement learning**, without any human demonstrations of step-by-step reasoning.

DeepSeek-R1-Zero (the pure RL version):

- Base model: DeepSeek-V3-Base (MoE LLM, 671B total parameters).
- **No supervised fine-tuning** before RL — trained directly with GRPO.
- Only **rule-based rewards**: (1) accuracy (is the boxed answer correct?) and (2) format compliance (did the model use `<think>...</think>`?).

GRPO: Group Relative Policy Optimisation. Originally proposed in DeepSeekMath [15]. GRPO eliminates the need for a separate critic/value model (halving memory requirements vs. PPO):

1. For each question q, sample a group of G outputs $\{o_1, \ldots, o_G\}$ from the current policy.
2. Compute reward r_i for each output.
3. Estimate advantage by **normalising within the group**:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_1, \ldots, r_G\})}{\text{std}(\{r_1, \ldots, r_G\})} \tag{7}$$

4. Update policy with a clipped surrogate objective:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q,\{o_i\}}\left[\frac{1}{G}\sum_{i=1}^{G}\Big(\min(\rho_i \hat{A}_i,\ \text{clip}(\rho_i, 1\pm\epsilon)\hat{A}_i) - \beta\, D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\Big)\right] \tag{8}$$

   where $\rho_i = \pi_\theta(o_i|q)/\pi_{\theta_{\text{old}}}(o_i|q)$.

The "Aha Moment." During R1-Zero training, the model **spontaneously** developed self-reflection. At an intermediate checkpoint, it began producing text like:

> *"Wait, wait. Wait. That's an aha moment I can flag here."*

Nobody told the model to think step by step. Nobody provided examples of self-correction. Through pure trial and error, the model discovered that **pausing to reconsider improves its answers.** The emergent behaviours include: chain-of-thought reasoning, self-verification, reflection, and dynamic strategy switching.

Results:

Benchmark	R1-Zero	R1 (full)	o1-0912
AIME 2024 (pass@1)	71.0%	79.8%	74%
MATH-500	95.9%	97.3%	—
AIME 2024 (consensus@64)	86.7%	—	—

R1-Zero, trained with *zero* human reasoning demonstrations, matches OpenAI o1 on AIME 2024 with majority voting.

The full R1 pipeline (four stages):

1. **Cold-start SFT:** Thousands of curated long-CoT examples for readability.
2. **Reasoning-oriented RL:** GRPO with accuracy + language consistency rewards.
3. **Rejection sampling + SFT:** Sample reasoning traces via rejection sampling ($\sim$600K reasoning + 200K non-reasoning examples).
4. **Alignment RL:** Final RL for helpfulness and harmlessness.

Distillation: R1's reasoning was distilled into smaller models with remarkable effectiveness:

Distilled model	AIME 2024	MATH-500
Qwen-1.5B	28.9%	83.9%
Qwen-7B	55.5%	92.8%
Qwen-32B	72.6%	94.3%
Llama-70B	70.0%	94.5%

The 1.5B distilled model outperforms GPT-4o on maths benchmarks.

4.4 Budget Forcing: Controlling How Long a Model Thinks

Muennighoff et al. [17] demonstrated that **1,000 curated training examples** are sufficient to unlock test-time scaling.

4.4.1 The s1K Dataset

Starting from 59,029 samples, curated through three-stage filtering:

1. **Quality filtering** (59K $\rightarrow$ 51.5K): Remove API errors, formatting issues.
2. **Difficulty filtering** (51.5K $\rightarrow$ 24.5K): Remove problems both Qwen-7B and Qwen-32B solve correctly. Use reasoning trace length as difficulty proxy.
3. **Diversity filtering** (24.5K $\rightarrow$ 1K): Classify into 50 domains; sample uniformly across domains, favouring longer traces.

4.4.2 How Budget Forcing Works

Two operations controlling test-time compute:

- **Shorten thinking (enforce maximum):** Forcefully insert the end-of-thinking token delimiter and "Final Answer:" to terminate reasoning early.
- **Extend thinking (enforce minimum):** When the model attempts to generate the end-of-thinking token, **suppress that token** and instead append the word **"Wait"** to the generation. This causes the model to continue reasoning, often leading it to **double-check its work and fix errors**.

The "Wait" token can be appended repeatedly (2×, 4×, 6×) for progressively deeper thinking. However, too many suppressions can cause repetitive loops.

4.4.3 Results

Model	AIME 2024
o1-preview	44.6%
s1-32B (no budget forcing)	50.0%
s1-32B (Wait 2×)	53.3%
s1-32B (Wait 6×)	57.0%

A 32B model fine-tuned on 1,000 examples (26 minutes on 16 H100 GPUs) beats o1-preview by up to 27% on competition maths. The authors invoke the **Superficial Alignment Hypothesis**: the model already acquired reasoning capabilities during pretraining; the 1,000-example SFT stage merely *activates* latent capabilities.

4.5 Taxonomy of Test-Time Compute Methods

4.5.1 The Snell et al. Framework (ICLR 2025)

Snell et al. [18] (Oral) provided the most rigorous framework for understanding when and how to scale test-time compute. Given a question q with ground-truth answer $y^*(q)$ and a compute budget N:

$$\theta^*_{q,y^*(q)}(N) = \arg\max_{\theta} \; \mathbb{E}_{y\sim\text{Target}(\theta,N,q)}\left[\mathbf{1}\{y = y^*(q)\}\right] \tag{9}$$

The optimal strategy is **prompt-dependent**: different questions call for different strategies. Compute-optimal allocation yields >**4**× **efficiency** over uniform best-of-N.

Difficulty-dependent results:

- **Easy problems:** Sequential revision is most effective.
- **Medium problems:** Beam search against a Process Reward Model (PRM) shows consistent gains.
- **Hard problems:** Neither strategy helps much — a larger pretrained model is needed.

The headline result: A small model with optimal test-time compute can outperform a **14**× **larger model** — but only on medium-difficulty problems.

4.5.2 Full Taxonomy

Category	Methods	Mechanism
Search-based	Best-of-N	Generate N solutions, select highest-scoring
	Beam search	Expand and prune partial paths via verifier
	Tree of Thoughts	BFS/DFS with LLM self-evaluation
	Self-Consistency	Multiple paths + majority vote
Refinement-based	Self-correction	Generate, critique, revise
	Budget forcing	Suppress stop tokens, inject "Wait"
	Sequential revision	Multiple rounds of refinement
Verification-based	ORMs	Outcome Reward Models (score final answer)
	PRMs	Process Reward Models (score each step)
	Generative PRMs	Use long-CoT model to verify steps
Hybrid / Learned	o1/o3 (OpenAI)	RL-trained internal reasoning tokens
	DeepSeek-R1	GRPO-trained extended CoT
	MCTS + PRM	Monte Carlo Tree Search guided by PRM

4.5.3 Process Reward Models

Lightman et al. [14] (OpenAI, 2023) showed that **process supervision** (scoring each reasoning step) significantly outperforms **outcome supervision** (scoring only the final answer). They released PRM800K (800,000 step-level human labels) and achieved 78% on a MATH test subset — demonstrating that fine-grained verification is a powerful complement to generation.

4.5.4 The System 1 / System 2 Analogy

The connection to Kahneman's dual-process theory:

- **System 1** (fast, intuitive): Standard LLM inference — a single forward pass, pattern matching.
- **System 2** (slow, deliberate): Test-time compute scaling — extended reasoning, search, verification.

4.5.5 Two Axes of Scaling: Strategic Implications

The capability frontier now has two dimensions: **train-time compute** (one-time capital expense) and **test-time compute** (per-query operating expense).

- **High-volume, low-latency tasks** (autocomplete, classification): Invest in a fast, large model. Test-time compute overhead is too costly at scale.
- **High-stakes, low-volume tasks** (complex analysis, planning, coding): Invest in a smaller model with extended thinking.
- **Difficulty matters:** Test-time compute is most effective for medium-difficulty problems.

5 Topic D: Grokking

5.1 The Discovery: Power et al. (2022)

Power et al. [19] (OpenAI, ICLR 2022 Workshop) discovered that neural networks trained on small algorithmic datasets can exhibit **delayed generalisation**: the model memorises the training data quickly, then — orders of magnitude later in training — suddenly generalises to the test set.

5.1.1 Experimental Setup

- **Tasks:** Binary operations on $\mathbb{Z}/97\mathbb{Z}$ (modular arithmetic with prime $p = 97$): addition, subtraction, division, polynomials ($x^2 + y^2$, $x^2 + xy + y^2$, $x^3 + xy$, etc.), and composition in the symmetric group S_5.
- **Data format:** Sequences "$a \circ b = c$" where c is masked and must be predicted.
- **Train/test split:** 50% of all (a, b) pairs for training; remainder for validation. A critical threshold exists between 30–40% — near 25% of data, a 1% decrease in training fraction leads to a 40–50% increase in median time to generalisation.
- **Architecture:** 2-layer decoder-only transformer, embedding dimension 128, 4 attention heads, $\sim 4 \times 10^5$ non-embedding parameters.
- **Optimiser:** AdamW, learning rate 10^{-3}, weight decay $\lambda = 1$ (notably large), $\beta_1 = 0.9$, $\beta_2 = 0.98$.

5.1.2 Training Dynamics (Division mod 97, 50% data)

Phase	Optimisation steps	What happens
Memorisation	$< 10^3$	Training accuracy $\to$ 100%
Stall	10^3–10^5	No visible improvement in test accuracy
Generalisation	$\sim 10^6$	Test accuracy $\to$ 100%

The gap between memorisation and generalisation spans roughly **three orders of magnitude** in optimisation steps. Every practitioner in the field would have stopped training during the stall phase.

5.1.3 Which Operations Grokked

All tested operations generalised except one: $x^3 + xy^2 + y \pmod{97}$ did not generalise within the optimisation budget at any training fraction up to 95%. Symmetric operations (e.g., $x + y$, $x^2 + y^2$) required less data to grok than asymmetric ones.

5.1.4 The Role of Regularisation

Weight decay was the single most important factor — it more than halved the data fraction needed for generalisation. Full-batch gradient descent was less effective than minibatch SGD, suggesting that stochastic noise also plays a role.

5.2 Mechanistic Explanation: Nanda et al. (ICLR 2023)

Nanda et al. [20] (ICLR 2023 Oral) fully **reverse-engineered the algorithm** learned by a small transformer trained on modular addition, revealing that the network discovers **discrete Fourier analysis**.

5.2.1 Setup

- **Task:** $(a + b) \bmod 113$ (prime modulus).
- **Architecture:** 1-layer transformer, embedding dimension $d = 128$, 4 attention heads (each $d/4 = 32$ dimensions), MLP hidden layer with 512 neurons, ReLU activation.
- **Training data:** 30% of all $113 \times 113 = 12{,}769$ input pairs ($\approx$3,831 training pairs).
- **Optimiser:** AdamW, lr $= 10^{-3}$, $\lambda = 1$, trained for 40,000 epochs.

5.2.2 The Learned Algorithm: Discrete Fourier Transform

The network learns to compute $(a+b) \bmod 113$ using five key frequencies $k \in \{14, 35, 41, 42, 52\}$ with angular frequencies $\omega_k = 2\pi k/113$. The algorithm proceeds in four stages:

Stage 1 — Embedding (W_E): The embedding matrix maps each integer $a \in \{0, \ldots, 112\}$ to its Fourier components:

$$a \longmapsto \big(\sin(\omega_k a),\ \cos(\omega_k a)\big) \quad \text{for each key frequency } k. \tag{10}$$

The embedding is **sparse in the Fourier basis**: only $\sim$6 frequencies have significant norm.

Stage 2 — Composition via trigonometric identities (Attention + MLP): The network computes:

$$\cos\big(\omega_k(a+b)\big) = \cos(\omega_k a)\cos(\omega_k b) - \sin(\omega_k a)\sin(\omega_k b) \tag{11}$$

$$\sin\big(\omega_k(a+b)\big) = \sin(\omega_k a)\cos(\omega_k b) + \cos(\omega_k a)\sin(\omega_k b) \tag{12}$$

This is the standard product-to-sum identity, implemented by the attention heads and MLP.

Stage 3 — Unembedding (W_U): For each candidate output $c \in \{0, \ldots, 112\}$, the network computes:

$$\cos\big(\omega_k(a+b-c)\big) = \cos(\omega_k(a{+}b))\cos(\omega_k c) + \sin(\omega_k(a{+}b))\sin(\omega_k c) \tag{13}$$

Stage 4 — Constructive interference: The logit for the correct answer $c^* = (a+b) \bmod 113$ receives constructive interference from all five key frequencies (all cosine terms equal 1 when $a + b - c^* \equiv 0 \pmod{113}$), while incorrect answers receive destructive interference. This gives c^* the largest logit.

5.2.3 Quality of the Approximation

The neuron-logit map $W_L = W_U W_{\text{out}}$ is approximately rank 10 and can be decomposed as:

$$W_L \approx \sum_k \big[\cos(\omega_k)\,\mathbf{u}_k^\top + \sin(\omega_k)\,\mathbf{v}_k^\top\big]$$

with residual Frobenius norm under **0.55%** of $\|W_L\|_F$. The final logit approximation explains $\sim$95% of variance.

5.2.4 Three Training Phases

Phase	Epochs	Characteristics
Memorisation	0–1,400	Train/excluded loss decline; test loss high
Circuit formation	1,400–9,400	Generalising circuit grows; $\|W\|_2$ falls
Cleanup	9,400–14,000	Memorising circuit pruned; test loss drops sharply

5.2.5 Progress Measures

Nanda et al. defined continuous metrics that track the formation of the generalising circuit:
Definition 5.1 (Restricted Loss)**.** Set all Fourier logit terms to zero except (a) the constant term and (b) the 20 terms corresponding to $\cos(\omega_k(a+b))$ and $\sin(\omega_k(a+b))$ for the five key frequencies. Then measure the resulting cross-entropy loss. This measures how well the Fourier circuit alone performs.
Definition 5.2 (Excluded Loss)**.** Remove only the key frequency components from the logits on the training data; measure the resulting training loss. This measures how well the memorising circuit performs *without* help from the generalising circuit.

Additional metrics include the **Gini coefficient** of Fourier component norms (measuring sparsity) and the L_2 norm of weights (tracking weight decay pressure). Crucially, these progress measures show **continuous, monotonic improvement** throughout training — even while test accuracy remains at chance. Grokking appears sudden on the test accuracy curve but is gradual in the underlying representation.

5.2.6 The Role of Weight Decay

Weight decay creates a pressure that penalises the high-norm memorising solution and favours the more parameter-efficient generalising (Fourier) solution. Without weight decay, no grokking occurs. The memorising and generalising circuits have **different norm-to-performance trade-offs**, and weight decay selects for the more efficient one.

5.3 Circuit Efficiency: Varma et al. (ICLR 2025)

Varma et al. [21] (Google DeepMind) provided a unifying theoretical framework for grokking based on **circuit efficiency**.

5.3.1 The Efficiency Framework

The total loss decomposes into two competing terms:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2 \tag{14}$$

where $\mathcal{L}_{\text{CE}}$ is the cross-entropy loss and λ is the weight decay coefficient.
Definition 5.3 (Circuit Efficiency)**.** The efficiency of a circuit is the **logit-to-parameter-norm ratio**: the magnitude of output logits achievable per unit of L_2 parameter norm. A circuit is more efficient if it achieves the same cross-entropy loss with lower parameter norm.

5.3.2 Three Conditions for Grokking

Grokking occurs when:

1. The generalising circuit C_{gen} generalises well while the memorising circuit C_{mem} does not.
2. C_{gen} is **more efficient** than C_{mem} (larger logits per unit parameter norm).
3. C_{gen} is **learned more slowly** than C_{mem}.

The key empirical finding: C_{mem}'s efficiency **decreases** with larger training datasets (memorisation cost grows with data), while C_{gen}'s efficiency **remains constant** (the generalising algorithm's parameter cost is fixed). This creates a critical dataset size D_{crit} where the efficiencies cross over.

5.3.3 Ungrokking

Definition 5.4 (Ungrokking)**.** A network that has already grokked (achieved perfect generalisation) **regresses to near-chance test accuracy** — a reversal of grokking.

To trigger ungrokking: take a network that has grokked on a large dataset (above D_{crit}), then retrain it on a significantly smaller dataset (below D_{crit}). Because C_{mem} is now more efficient, gradient descent with weight decay shifts the network back toward memorisation.

5.3.4 Semi-Grokking

Near D_{crit}, where both circuits have comparable efficiency, the network plateaus at **partial** test accuracy rather than completing the transition to perfect generalisation.

5.4 Grokking at Scale: Li et al. (2025)

Li et al. [22] demonstrated that grokking occurs during **real LLM pretraining** at the 7B-parameter scale, confirming that it is not merely a toy phenomenon.

5.4.1 Model and Setup

- **Model:** OLMoE-1B-7B — decoder-only transformer with Mixture-of-Experts (MoE), 7B total parameters, ∼1.3B active per forward pass. 64 routed experts per layer, top-8 selected per token.
- **Training data:** ∼5.1 trillion tokens.

5.4.2 Local Grokking

Unlike the global, synchronous grokking in toy settings, LLM pretraining exhibits **asynchronous, domain-level** grokking:

- Different domains (maths, code, commonsense) are memorised at different training steps.
- Each domain takes varying amounts of additional training to generalise.
- The training loss has converged globally, but downstream benchmark performance continues to improve.

5.4.3 Pathway Evolution Metrics

The MoE architecture provides a natural lens. Li et al. defined:

1. **Edit Distance:** Levenshtein distance between different samples' expert routing sequences. During grokking, this **declines** — pathways become more structured.
2. **Pathway Consistency:** Cosine similarity between consecutive layers' expert embeddings. This **increases** during grokking — routing becomes smoother.

These metrics showed correlation $r > 0.97$ with downstream test accuracy, while training loss showed weak, inconsistent correlations. Four domains demonstrated grokking: arithmetic reasoning, code generation, commonsense QA, and domain-specific QA.

5.5 Connection to Classical Learning Theory

5.5.1 Violation of the Bias-Variance Tradeoff

In classical statistics, a model that has overfit (high variance, low bias) is expected to *stay* overfit. Grokking directly contradicts this: the model first interpolates the training data perfectly, then

transitions to perfect generalisation under continued training. This should not happen under classical assumptions.

5.5.2 Connection to Double Descent

Double descent [23] describes a non-monotonic test error curve: classical U-shape, peak at the interpolation threshold, then improvement in the overparameterised regime. Davies [24] proposed that **grokking and double descent are instances of the same phenomenon**: inductive biases (weight decay) prefer better-generalising but slower-to-learn patterns.

Three pattern types in the unified framework:

1. **Heuristics:** Fast-learning, well-generalising (Type 1).
2. **Memorisation:** Fast-learning, poorly-generalising (Type 2).
3. **Slow generalising:** Slow-learning, well-generalising, ultimately preferred by inductive biases (Type 3).

Grokking = **epoch-wise** transition from Type 2 to Type 3 dominance.
Double descent = the same transition viewed as a function of **model capacity**.

6 Synthesis: What the Audience Should Take Away

1. **Unpredictability of capabilities:** Emergent abilities mean we cannot reliably forecast what the next model will do. Both real emergence and metric artifacts contribute to this uncertainty, making strategic planning difficult.
2. **Chain-of-thought changed everything:** A simple prompting trick (2022) evolved into an entirely new paradigm of AI systems that reason. The trajectory from "Let's think step by step" to DeepSeek-R1's spontaneous self-reflection took less than three years.
3. **A new cost structure:** Test-time compute creates a second scaling axis. Budget forcing, self-consistency, and thinking models all trade inference-time compute for accuracy — creating deployment decisions that depend on task difficulty, volume, and latency tolerance.
4. **Delayed understanding:** Grokking shows that models can develop capabilities long after training loss has converged. For safety-critical deployments, post-training evaluation is not a one-time event.
5. **The honest answer to "do we understand these systems?":** Partially. We can reverse-engineer small models (Nanda's Fourier circuits). We can empirically characterise scaling behaviour (Chinchilla, Snell). But we cannot predict emergence, and we do not fully understand why reasoning emerges from pure RL. The frontier of understanding is advancing, but it lags behind the frontier of capability.

References

[1] P.W. Anderson. More Is Different. *Science*, 177(4047):393–396, 1972.

[2] J. Kaplan, S. McCandlish, T. Henighan, T.B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling Laws for Neural Language Models. *arXiv:2001.08361*, 2020.

[3] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, et al. Training Compute-Optimal Large Language Models. *NeurIPS*, 2022.

[4] T. Besiroglu, E. Erdil, M. Barnett, and J. You. Chinchilla Scaling: A Replication Attempt. *arXiv:2404.10102*, 2024.

[5] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, et al. Emergent Abilities of Large Language Models. *Transactions on Machine Learning Research (TMLR)*, 2022.

[6] S. Lin, J. Hilton, and O. Evans. TruthfulQA: Measuring How Models Mimic Human Falsehoods. *ACL*, 2022.

[7] R. Schaeffer, B. Miranda, and S. Koyejo. Are Emergent Abilities of Large Language Models a Mirage? *NeurIPS*, 2023. Outstanding Paper Award.

[8] L. Berti, F. Giorgi, and G. Kasneci. Emergent Abilities in Large Language Models: A Survey. *arXiv:2503.05788*, 2025.

[9] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q.V. Le, and D. Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS*, 2022.

[10] T. Kojima, S.S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large Language Models are Zero-Shot Reasoners. *NeurIPS*, 2022.

[11] X. Wang, J. Wei, D. Schuurmans, Q.V. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR*, 2023.

[12] S. Yao, D. Yu, J. Zhao, I. Shafran, T.L. Griffiths, Y. Cao, and K. Narasimhan. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *NeurIPS*, 2023.

[13] D. Zhou, N. Scharli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q.V. Le, and E. Chi. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. *ICLR*, 2023.

[14] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's Verify Step by Step. *arXiv:2305.20050*, 2023.

[15] DeepSeek-AI. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024.

[16] DeepSeek-AI, D. Guo, D. Yang, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*, 2025. Published in *Nature*.

[17] N. Muennighoff, Z. Yang, W. Shi, X.L. Li, L. Fei-Fei, et al. s1: Simple Test-Time Scaling. *arXiv:2501.19393*, 2025.

[18] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters. *ICLR*, 2025. Oral.

[19] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets. *ICLR Workshop*, 2022.

[20] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress Measures for Grokking via Mechanistic Interpretability. *ICLR*, 2023. Oral.

[21] V. Varma, R. Shah, Z. Kenton, J. Kramar, and R. Kumar. Explaining Grokking Through Circuit Efficiency. *ICLR*, 2025.

[22] Z. Li, C. Fan, and T. Zhou. Where to Find Grokking in LLM Pretraining? *arXiv:2506.21551*, 2025.

[23] M. Belkin, D. Hsu, S. Ma, and S. Mandal. Reconciling Modern Machine Learning Practice and the Bias-Variance Trade-off. *PNAS*, 116(32):15849–15854, 2019.

[24] X. Davies, L. Langosco, and D. Krueger. Unifying Grokking and Double Descent. *arXiv:2303.06173*, 2023.